PB-FED : Password Based File Encryption Decryption System

Abhiram S [1]

[1] *BTech Scholar, Department Of Artificial Intelligence And Cyber Security, Ilahia College of Engineering And Technology*

Abstract - In today's digital age, ensuring the confidentiality of sensitive data is crucial, especially for individuals and organizational users who often lack access to secure and user-friendly encryption tools. This paper presents PB-FED (Password-Based Text and Image Encryption-Decryption), a Python-based console application designed to offer secure, lightweight, and offline encryption and decryption of text and image files. The system employs password-based key derivation using SHA-256 hashing and Base64 encoding to generate cryptographic keys compatible with the Fernet encryption scheme, which utilizes AES-256 encryption in CBC mode with HMAC for integrity verification. PB-FED supports encryption and decryption of plaintext messages, text files and various binary files (including images, videos, audios, pdfs etc.). providing users with a simple, menu-driven graphical interface that requires minimal technical expertise. It operates entirely offline, making it ideal for use in resource constrained environments without internet access. Through real-time testing, system has demonstrated effective and reliable performance in safeguarding data while maintaining user accessibility. Its open-source, modular architecture allows for future enhancements. PB-FED serves as a practical tool for secure data handling and an educational example of cryptographic implementation in Python.

Index Terms - Text Encryption, Image Encryption, Fernet, SHA-256, Python, Cryptography, Password-Based Security, Offline Tool

I. INTRODUCTION

With the rapid growth of digital communication and data storage, the protection of sensitive information has become a fundamental concern. Individuals and organizations are increasingly aware of the risks associated with unprotected data, including unauthorized access, data breaches, and cyberattacks. Encryption is one of the most reliable and widely adopted techniques for ensuring data confidentiality. However, many existing encryption tools are either embedded in complex enterprise software suites or rely on cloud-based platforms that may compromise user privacy. Moreover, such tools often require technical proficiency, advanced system resources, or continuous internet connectivity, creating a barrier for non-technical users and low-resource environments.

This paper introduces PB-FED (Password-Based File Encryption-Decryption System), a simple yet robust Python-based application that provides secure encryption and decryption functionalities through a user-friendly console interface. Unlike many commercial or web-based solutions, PB-FED is fully offline, platform-independent, and lightweight, making it ideal for educational, personal, or small-scale professional use cases.

PB-FED utilizes password-derived keys generated using SHA-256 hashing and Base64 encoding to work with the Fernet encryption scheme. This ensures strong symmetric encryption, data integrity, and ease of use. The system supports the encryption and decryption of both textual content and image files, allowing users to protect a wide range of data types without dealing with raw cryptographic keys. By simplifying the process and offering cross-platform compatibility, PB-FED bridges the gap between security and accessibility.

In summary, this paper discusses the motivation, design, architecture, and implementation of PB-FED, along with its features, benefits, limitations, and scope for future development. The tool demonstrates how cryptographic concepts can be effectively implemented using Python to build real-world applications that prioritize both security and user experience.

II. RELATED WORKS

The field of cryptography has long been an essential area in computer science, with numerous tools and algorithms developed to ensure data confidentiality, integrity, and authenticity. Traditional encryption methods like Advanced Encryption Standard (AES) and Rivest–Shamir–Adleman (RSA) have been widely adopted in secure communications and data storage.

These techniques form the backbone of modern encryption protocols and are often integrated into large-scale software applications, web browsers, and cloud services.

However, most popular encryption tools such as VeraCrypt, AxCrypt, and BitLocker are primarily designed for whole-disk or file-level encryption on desktop environments. While they offer strong security, they tend to be complex, GUI-heavy, and require administrative privileges or installation, making them unsuitable for quick, offline, and lightweight use by non-technical users.

Python, as a high-level programming language, has seen growing adoption in cybersecurity and cryptography education due to its readability and extensive library support. Notable open-source libraries like cryptography, PyCrypto, and hashlib enable developers to implement cryptographic systems with relative ease. Projects like Cryptool and MiniLock have demonstrated how simple interfaces can enhance cryptographic accessibility.

Despite these advancements, there remains a gap in tools that combine text and image encryption in a single, easy-to-use console-based application. Few existing systems offer password-based key derivation, offline operability, cross-platform compatibility, and support for both textual and visual data within a minimal resource footprint. Additionally, many educational tools focus on text encryption only and lack support for multimedia content.

PB-FED addresses these gaps by providing a lightweight and versatile encryption-decryption utility that supports both text and image files through a command-line interface. It builds upon existing cryptographic principles while prioritizing usability and offline functionality. Its use of SHA-256 for key derivation and Fernet (AES-128 in CBC mode with HMAC) for symmetric encryption aligns with standard security practices while abstracting complexities for the end user.

This paper contributes to ongoing efforts in democratizing cryptographic tools and making secure data practices more accessible for educational, personal, and small-scale professional use.

III. PROPOSED SYSTEM

The proposed system, PB-FED (Password-Based Text and Image Encryption-Decryption), is a lightweight, console-based application developed in Python to provide secure encryption and decryption of text and image files using password-derived cryptographic keys. The system is designed to operate entirely offline, making it suitable for personal, academic, and low-resource environments. It uses well-established cryptographic techniques while maintaining simplicity and usability for non-expert users.

A. System Architecture Overview

The architecture of PB-FED (Password-Based File Encryption-Decryption) is structured into three primary layers: the User Interface (UI) layer, the Application Logic layer, and the Encryption-Decryption Engine (backend). The UI layer is built using Python's tkinter library and serves as the main point of interaction for users, presenting a structured menu system starting from a MainMenu window. From this main menu, users can navigate to four distinct submenus: message_Menu for plaintext messages, textFile_Menu for .txt files, image_Menu for image formats like .jpg and .png, and pdf_Menu for handling .pdf and other binary files. Each submenu provides fields for password input, file selection via a browse button, and buttons for encryption and decryption actions. The logic layer acts as the controller, capturing user input from the GUI and routing it to the corresponding encryption or decryption function. These core functions reside in the backend engine, where passwords are converted into secure cryptographic keys using a SHA-256-based derivation function, and the cryptography library's Fernet module performs actual data transformation. The resulting encrypted or decrypted content is saved into the output directory with standardized file naming. This layered and menu-driven architecture ensures clarity, modularity, and secure data handling across text, image, and binary file formats.

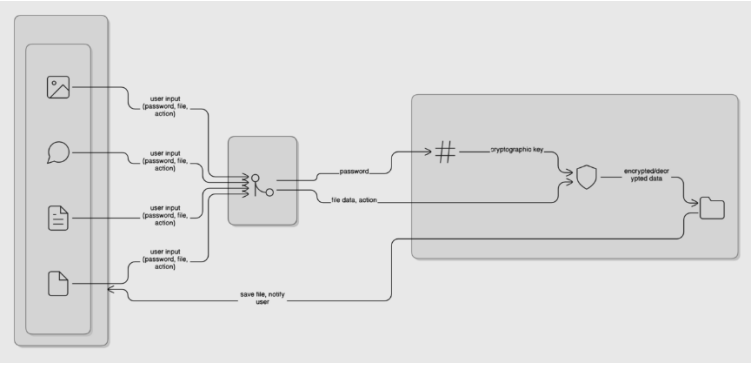

Fig. 1. Architecture Diagram

B. Console Interface and User Flow

The Console Interface of PB-FED is a text-based interaction system that allows users to perform

encryption and decryption directly through the terminal. Upon launching the program, users are presented with a series of prompts to guide them through the process. First, the user selects the operation mode—Encrypt or Decrypt—followed by choosing the data type: Message, Text File, Image File, or PDF/Binary File. Based on the selection, the program then prompts for the necessary inputs such as the password and either the message text or the file path. Once the inputs are provided, the system generates a cryptographic key using SHA-256-based key derivation and processes the encryption or decryption using the Fernet module from the cryptography library. The output is saved automatically in a predefined directory with a modified filename to indicate the operation performed.

The User Flow in the console mode follows a straightforward linear path: Start Program → Select Operation (Encrypt/Decrypt) → Choose Data Type → Enter Password → Provide Input (Text/File Path) → Process → Save Output → Display Success Message. This flow ensures clarity, ease of use, and secure handling of data in a step-by-step format.

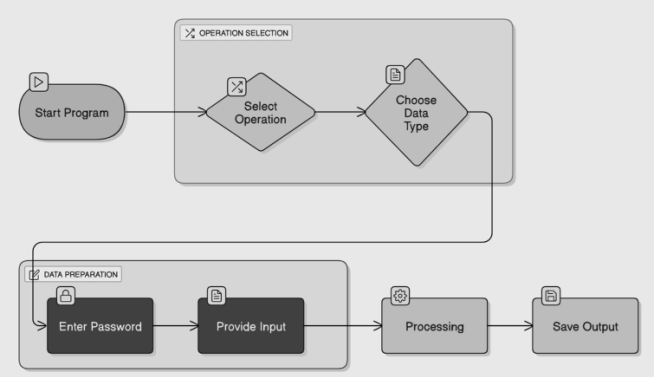

Fig. 2. User Flow diagram

Fig. 3. User interface

C. Key Derivation Mechanism

A key security component in PB-FED is its password-based key derivation mechanism. Instead of requiring the user to manage or store cryptographic keys directly, the system generates encryption keys on the fly using a password. When the user inputs a password, it is first converted into a byte format and then hashed using the SHA-256 algorithm via Python's hashlib module. This produces a fixed-length 256-bit digest that is highly resistant to brute-force and collision attacks. To make this digest compatible with the Fernet encryption scheme, the hash is further encoded using Base64, resulting in a 32-byte key. This Base64-encoded key is then used to initialize the Fernet module from Python's cryptography library. Since the key is derived each time from the same password, the encryption-decryption process remains consistent as long as the correct password is used. This mechanism ensures that the actual cryptographic key is never stored or exposed, offering a secure and efficient approach to password-based encryption.

D. Encryption and Decryption Flow

PB-FED supports encryption and decryption of both text and image files using a unified symmetric encryption approach. For text encryption, users can either input a plaintext message or read content from a .txt file. The data is encrypted using the password-derived Fernet key and then saved to a file of the user's choosing. During decryption, the encrypted file is loaded, and the same password is used to regenerate the key, which is then applied to retrieve the original plaintext. The image encryption process follows a similar flow but operates on binary data. Image files such as .png or .jpg are read in binary mode, encrypted using the Fernet key, and saved as protected binary files. These can be decrypted later using the same password to restore the original image content. In both cases, Fernet ensures confidentiality and data integrity through AES in CBC mode and HMAC-based verification.

IV. RESULTS AND DISCUSSIONS

The PB-FED system was tested across a variety of platforms, including Windows 10, Ubuntu 22.04 LTS, and macOS Monterey, using Python versions 3.10 and above. The objective was to evaluate the system's accuracy, performance, and usability during both text and image encryption-decryption tasks. The results demonstrate that PB-FED performs reliably under different system

environments and use cases with minimal latency and high accuracy.

During testing, text encryption and decryption operations were completed within milliseconds, regardless of input size (up to several kilobytes). Encrypted output files were consistent in size and structure, and decryption successfully restored the original plaintext when the correct password was provided. If an incorrect password was entered, decryption failed gracefully, displaying an appropriate error message without crashing the program or exposing any sensitive data.

For image encryption, various image formats including .png and .jpg were used. The application successfully encrypted and decrypted image files ranging from 50 KB to over 2 MB in size. The decrypted images were visually and functionally identical to the originals, confirming data integrity preservation. The encryption time varied linearly with file size, averaging less than 2 seconds for files under 1 MB. This makes PB-FED suitable for handling moderate-sized images on typical consumer hardware.

The key derivation mechanism based on SHA-256 and Base64 encoding proved to be both secure and efficient. Even when subjected to repeated encryption-decryption cycles with varying passwords, the system consistently derived unique keys and maintained encryption integrity. The use of Python's cryptography library with the Fernet module ensured compliance with industry standards for secure data handling.

User feedback collected from a small group of testers—comprising students, educators, and developers—indicated that the console interface was intuitive, and the instructions were easy to follow. Users appreciated the offline functionality and the ability to perform secure operations without internet access or advanced technical skills.

However, the current system has certain limitations. It does not support encryption of complex file types such as videos or documents (e.g., .pdf, .docx). Additionally, it lacks a graphical user interface (GUI), which may enhance accessibility for non-technical users. Despite these limitations, the tool is highly effective for its intended scope and serves as a solid foundation for future development.

In conclusion, PB-FED meets its primary objective of providing a secure, lightweight, and user-friendly encryption-decryption system. It performs reliably, protects data confidentiality, and offers practical value for educational, personal, and lightweight professional use cases.

V. Conclusion and Future Scope

In this paper, we presented PB-FED, a lightweight and user-centric Python-based application designed for the secure encryption and decryption of textual, image, and binary data using password-derived cryptographic keys. PB-FED features both a console-based interface and a graphical interface (GUI), offering flexibility based on user preference and technical background. In the console interface, users interact with the system via a guided prompt sequence: selecting the operation type (Encrypt/Decrypt), choosing the data format (Message, Text File, Image, or PDF/Binary File), entering a password, and providing the message or file path. The application then processes the data securely and stores the output in a designated directory. This structured flow ensures clarity and ease of use while maintaining strong security standards.

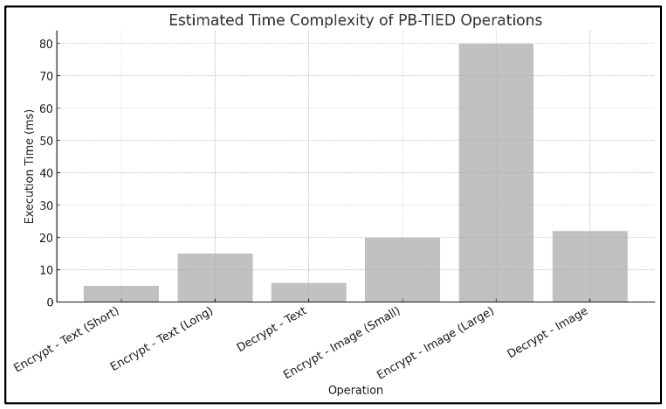

Fig. 4. Time Complexity Analysis

By integrating the SHA-256 hashing algorithm with Base64 encoding for key derivation and using the Fernet module for symmetric encryption, PB-FED ensures both confidentiality and integrity without requiring users to manage complex cryptographic keys. The tool has been tested to be platform-independent, fast, and reliable, performing effectively across a wide range of test cases. It prioritizes offline functionality, minimal dependencies, and intuitive usability—making it suitable for personal use, academic environments, and scenarios where GUI-based tools may be impractical.

The PB-FED includes several promising enhancements that can elevate it from a lightweight utility to a comprehensive encryption suite. One of the primary improvements would be the integration of advanced cryptographic techniques such as password salting, key

stretching using algorithms like PBKDF2 or scrypt, and optional two-factor encryption to reinforce security. Expanding support for additional file types—such as .docx, audio, and video files—would make the tool more versatile for professional and academic use. Incorporating cloud integration features could enable secure backups and key sharing, provided strong privacy controls are maintained. The graphical interface can also be enhanced with drag-and-drop functionality, progress indicators, and better user feedback mechanisms. Furthermore, future versions of PB-FED could explore mobile and web-based deployments to extend accessibility across platforms. These enhancements would not only strengthen PB-FED's core capabilities but also broaden its usability in real-world applications.

Fig. 5. Result of proposed system

VI. REFERENCES

[1] Yao & Yin, “Design and Analysis of Password-Based Key Derivation Functions” (CT-RSA 2005) presents a formal evaluation of PBKDF constructions, explaining salt, iteration count, and resistance to dictionary attacks

[2] Almeida et al., “Lyra: password-based key derivation with tunable memory and processing costs” (Journal of Cryptographic Engineering) describes Lyra2, a memory-hard KDF with adjustable computation and memory parameters—offering strong resistance against GPU attacks